

Exam: B188633

Date: May 2, 2022

Two-Dimensional Decomposition of Percolate Message Passing Parallel Program (Part 2)

Course: Message Passing Programming

Course Organiser: Dr David Henty

Assessment: 2



THE UNIVERSITY *of* EDINBURGH

Table of Contents

1. Introduction.....	2
2. Code Description.....	2
2.1 Two-dimensional decomposition.....	2
2.2 Non-blocking communications	3
2.3 New boundary conditions.....	3
2.4 Average map value.....	3
2.5 Stopping computation	3
2.6 Other changes.....	4
3. Correctness Testing	4
3.1 Method	4
3.2 Segmentation fault	5
3.2 Correctness test results	5
4. Performance Analysis.....	6
4.1 Method	6
4.2 Performance test results.....	7
5. Conclusions.....	8
References.....	9
Appendix A: Correctness Test Data	10
Appendix B: Performance Test Data	10

1. Introduction

This report is the continuation of a discussion of a solution to the percolate problem described in [1]. In the previous report (also interchangeably referred to as part 1), we discussed a plan to transform an existing parallel solution to the percolate problem using one-dimensional domain decomposition to a more advanced solution using two-dimensional decomposition [2]. We discussed how we can use the various features of the Message Passing Interface (MPI) to undertake communications. We detailed a structure of our program that allows for flexibility and for extensibility. Also, we discussed our method to test the program for correctness and for performance. In this report, we discuss changes to the design and implementation of the program from our original proposals. Also, we discuss correctness and performance tests of our program, along with an overall review of the project.

2. Code Description

In this section, we discuss changes to the design and implementation of the two-dimensionally decomposed program from our original proposals. Also, we discuss some successes and challenges of our current design.

2.1 Two-dimensional decomposition

In our previous report, we discussed a simple decomposition method where the system size (i.e., L) would be divided by the number of processes (i.e., $NPROC$) to determine the size of each dimension (i.e., M and N) of the smaller grid. For instance, a system size of 8 across 4 processes would mean that the value of M is 2 (i.e., $L/4$), and the value of N is 2 (i.e., $L/4$).

However, we realised later that this naïve method results in the wrong two-dimensional decomposition as the local grids of all processes (i.e., $M \times N \times NPROC$) do not cover the whole area of the system size (i.e., $L \times L$). To resolve this issue, we applied the **virtual cartesian topology** feature of MPI to create a two-dimensional grid to decompose the map accurately.

We used `MPI_Dims_create()` to allow MPI to choose an appropriate decomposition over a 2D cartesian grid. The advantage of using such a feature is that MPI attempts to balance the distribution by minimising the difference in the number of processes assigned to each dimension [3]. Moreover, this method enables a flexible decomposition as MPI can freely decompose odd numbers and prime numbers of processes. We only need to check whether both M and N dimensions divide system size L .

To illustrate some examples of the decomposition, suppose we have $L=8$ and $NPROC=8$, then we might have $M=L/4$ and $N=L/2$. When using 3 processes, for instance, a 768×768 problem size can be easily divisible into a 3×1 grid of 256×768 .

2.2 Non-blocking communications

Non-blocking communications have been implemented as described originally, i.e. using a combination of **MPI_Issend()** and **MPI_Recv()** for non-blocking halo swaps with the four neighbouring processes (i.e., left, right, down, and up). Four calls to **MPI_Issend()** are made consecutively to send the appropriate data to the neighbouring processes, followed by four calls to **MPI_Recv()** to receive the appropriate data from the neighbouring processes. The consecutive calls to non-blocking sends enables the program to overlap communication with computation, as the **MPI_Issend()** routines return immediately. After all the receives complete, **MPI_Waitall()** is called to ensure that all the non-blocking sends complete.

We drew our arrays in the coordinates format. Additionally, C language's layout in memory is such that **x[i][j]** is followed by **x[i][j+1]**. Hence, we created a vector datatype using **MPI_Type_vector()** to ensure the appropriate values in the main arrays are sent over as vectors to the neighbouring processes up and down. The neighbouring processes receive these vectors in temporary arrays, and then the processes copy the halo values to their appropriate locations in their main arrays.

2.3 New boundary conditions

The new boundary conditions were implemented as planned. As we used the virtual cartesian topology, we computed neighbouring processes with **MPI_Cart_shift()**, and simply set the horizontal dimension to be periodic when creating the new communicator with **MPI_Cart_create()**. Some region was required to be non-periodic in the horizontal dimension. We used the same logic that has been used in the serial code, i.e., zeroing the halos in the non-periodic region after the halo swaps complete.

2.4 Average map value

In part 1, we mentioned using **MPI_Allreduce()** for computing the average map value. This is fine, but we just used **MPI_Reduce()** as the value is just printed to the screen. At regular intervals (i.e. 100 iterations in our case), the local sums of the grids of all processes were added up with **MPI_Reduce()** to store a global sum in the root process. Then, the root process divided the global sum by the number of squares (i.e., $L \times L$) to determine the map average.

2.5 Stopping computation

In part 1, we mentioned that we would stop computation (i.e., main update loop) when the changes in a step converge to zero. In the previous solution using 1D decomposition, number of changes was computed using **MPI_Reduce()**. Hence, only the root process (i.e., rank 0) knew the total number of changes. However, to stop the computation correctly, all processes need to know the total number of changes. Thus, we used **MPI_Allreduce()** in the 2D decomposition code.

At the end of each iteration, we check whether the total number of changes in the current iteration is zero. If it is, the program prints the current map average, and then it breaks out of the while loop.

2.6 Other changes

In our previous report, we discussed the possibility of introducing additional changes to our code, if time permitted, for further flexibility and functionality. We introduced one such enhancement, where we allow the **rho** parameter (i.e., density of rock) to be optionally specified as a runtime argument. The default value is 0.4040 as required, so the program still runs with this value of rho if no particular value is specified.

3. Correctness Testing

In this section, we discuss our testing method that we applied to ensure that the parallel program runs correctly.

3.1 Method

In part 1, we discussed a formal method to test for correctness. Here is the list of data that we mentioned we would compare to confirm correctness:

- (1) Compare that the actual density is equal for both serial and parallel versions.
- (2) Compare that the algorithm takes the same number of steps to complete in both versions.
- (3) Verify whether the given configuration parameters result in the same answer (i.e., whether percolation occurs or not) in both versions.
- (4) Compare the cluster sizes of the largest 8 clusters in both versions.
- (5) Visually inspect the largest 8 clusters from the output images of both versions.

In the second phase of the project, we realised that we can also compare maps quantitatively as well as qualitatively. For instance, each square value of the final state of the map in the parallel program can be compared against the corresponding square value in the map of the serial program. We can compute a match percentage based on this comparison. If the match percentage is below a benchmark, say 99%, then it can raise a warning signal for us to investigate further for any possibility of errors.

Besides the formal testing method, some heuristics have been very helpful to detect bugs early during development. Each component in the program was informally tested with a debug print statement. For

instance, after zeroing all the halos of the main array during initialisation, we added up all the halo values in each process and printed the sum to check whether the sum is equal to zero in each process. After completing halo swaps, we compared a few values of appropriate process-pairs (e.g., halo values at the outer-right end or when $i=M+1$ of one process compared to its right neighbour's inner-left end values or when $i=1$) and compared the sums of halo values of appropriate process-pairs. When applying a reduction operation such as **MPI_Reduce()** for computing global sum, the local value of each process was printed to check whether their sum indeed equals the global sum computed by the reduction operation.

The limitations of our program have been checked as well. For instance, our program aborts with an error message when any of the dimensions (i.e., M or N) does not divide system size (i.e., L), indicating the user to try specifying another value of L . We tested this logic by running the program with out-of-scope values of L and checked if the error messages are being returned appropriately.

3.2 Segmentation fault

The current version of the code throws a segmentation fault on both the login and backend nodes of Cirrus [4]. Normally, you can increase the size of the stack (with **ulimit -s unlimited** command) to fix this issue. However, the code throws a segmentation fault after the percolate update loop completes, and during the map visualisation IO (i.e., **mapwrite**). Additionally, the result of the percolate algorithm (i.e., whether percolation occurs or not) is not reliable on the frontend nodes for some configurations of ρ , $NPROC$, and L .

Although the segmentation fault does occur on the backend nodes, the percolate algorithm results in the correct answer. So, we ran the current version of the code on the backend nodes for reliable correctness testing (i.e., without the map visualisation) and performance testing. A set of **.out** and **.err** files (with **JOB ID 2474583**) from a sample run of the parallel code on 2 backend nodes of Cirrus are included for reference in the supplementary code. The sample run used the configuration: ($L=768$, $NPROC=64$, $\rho=0.4040$).

We are currently working on debugging the subtle fault that occurs in the code. Our early ideas include reducing the use of several arrays in the code for copying data back and forth. With this approach, we might be able to reduce the memory footprint and check whether this change resolves the issue. We can use tools such as **GDB** and **Valgrind** to aid in our debugging process.

3.2 Correctness test results

The current version of our parallel program has been formally tested on eight different combinations of the configuration parameters of ρ , L , seed, and $NPROC$, and the results have been compared to the corresponding results using the same parameters (except $NPROC$) in the serial code. Due to the segmentation fault in the parallel code that impacts the map visualisation IO, we were not able to compare results of the **mapwrite** function as planned. The results of these tests are tabulated in Appendix A.

Overall, our program returned the correct answers (i.e., whether percolation occurs or not) in all tested configurations. Due to the flexibility of our decomposition method using the virtual cartesian topology feature in MPI, we can run our program with odd numbers and prime numbers of processes. Also, our program returns the correct results when tested with the edge cases of ρ (i.e., 0 and 1).

There is some variance in the number of steps between the serial and the parallel programs. This is due to the boundary conditions of both programs. In the serial version, only a few rows at the bottom end of the map remains non-periodic in the horizontal direction. However, in the parallel code, each local map of a process has a non-periodic region at the bottom of the map. This difference in the data leads to different states of the whole maps of the serial and parallel versions in each iteration, which results in some variance in the number of steps to complete the percolate algorithm.

4. Performance Analysis

In this section, we discuss the performance analysis of our two-dimensionally decomposed percolate parallel code.

4.1 Method

The program was compiled on Cirrus with -O3 optimisation using the Intel C Compiler (**intel-compilers-19**) and the Message Passing Toolkit (**mpt/2.25**) modules. We timed only the core update loop to compute average time per step as we are not particularly interested in the performance of the serial initialisation and IO sections. Besides, we cannot time the entire program from start to finish using the current code due to the segmentation fault described above.

The program was timed using calls to **MPI_Barrier()** and **MPI_Wtime()**, and then calculating the difference of the times between calls to **MPI_Wtime()**. The calculated time was simply divided by the number of steps to determine the time per step. We fixed the number of steps for each configuration of L . This was done so that for small system sizes the program runs for a few seconds at least to reduce any noise from the system. Consequently, for large system sizes, fixing number of steps avoids unnecessary burning of CPU time. We ran each test with the same configuration three times and computed the average time per step for that configuration. Also, standard deviation and 95% confidence interval of time per step were computed to confirm that the performance is stable and reproducible.

We ran our code using up to 4 compute nodes of Cirrus (up to 128 processes), and computed speedups and parallel efficiencies. These metrics enable us to analyse the parallel performance of the code. Also, we tested our code with four different system sizes to view the effects on scalability. The raw

performance data of a sample configuration of L , and the average time per step data of all tested system sizes have been made available in Appendix B.

4.2 Performance test results

We observe in Figure 1 that speedup generally increases (up to a point) with increasing number of processes. Our code does not scale linearly with the number of processes due to the parallel overheads, which include operations that do not benefit from parallelisation (e.g., reduction operations, halo swaps). This effect is expected from Amdahl's law. Consequently, we observe in Figure 2 that the parallel efficiency declines with higher number of processes.

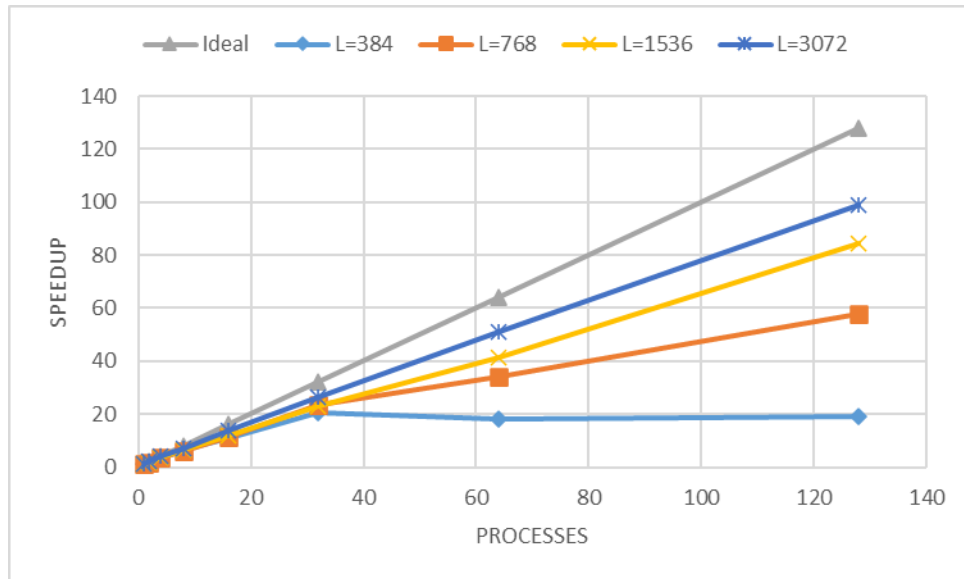


Figure 1: Speedups of the parallel code across a range of process counts and system sizes.

We observe in Figure 1 that speedup improves with larger system sizes and approaches towards the ideal speedup line. Also, we observe in Figure 2 that parallel efficiency improves with larger system sizes. This effect of higher scalability with higher system sizes is expected from Gustafson's law.

Overall, we conclude that our program is performing in line with our expectations. We do not compare the performance of our current parallel code with that of the one-dimensionally decomposed version, which was described in the previous report. This is because the earlier version was timed end-to-end, which includes initialisation and IO. So, it is not reasonable to compare the performances of both codes as there is no apples-to-apples comparison.

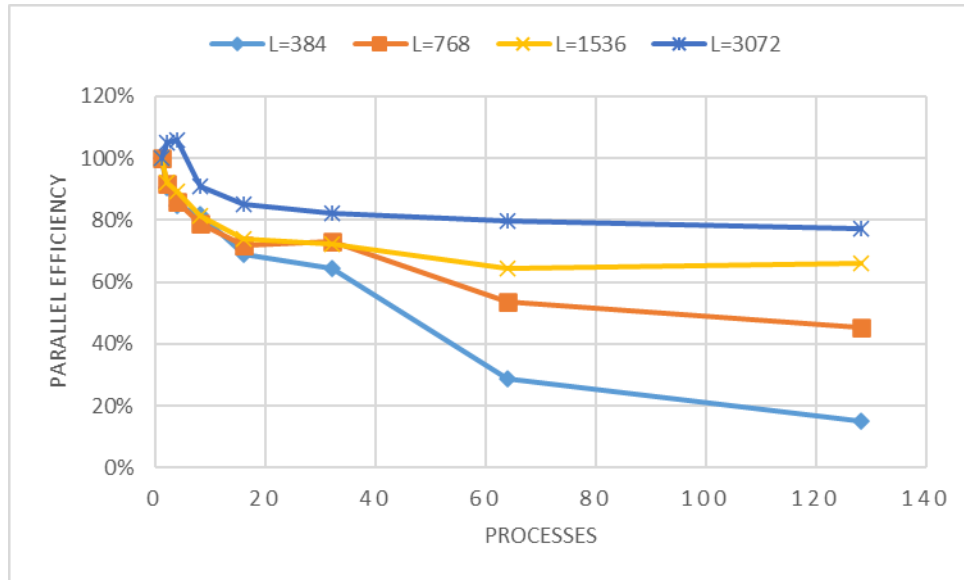


Figure 2: Parallel efficiencies across a range of process counts and system sizes.

5. Conclusions

Our goal in this report was to discuss a parallel algorithm to solve the percolate problem using two-dimensional decomposition. We discussed the essential components of our program, along with correctness testing and performance analysis of our code. After a review of our project, we believe there are further enhancements that could be made to our program to make it more performant, flexible and functional.

In our code, we discussed the usage of non-blocking communications to overlap communication with other communication (i.e., multiple immediate synchronous sends called consecutively). In the future, we can overlap communication with computation. For instance, the squares in the main array requiring halo values can be updated later after the halo swaps complete.

We can have a better structure of our program. For instance, the main function of our program looks monolithic due to the several operations involved in the program. We can introduce a modular design to our code, where we can have a separate library for communications, a separate library for the core percolate algorithm, a separate library for array operations (e.g., initialising arrays, copying data, zeroing squares), and such. With a modular design, we can easily apply a more robust testing method, e.g., unit testing the individual components with a unit testing framework.

In our code, we introduced the flexibility of specifying the `rho` parameter as an optional runtime argument. Additionally, we can enhance the code such that both `L` and `NPROC` can be accepted as optional command line arguments.

Our current program uses **`MPI_Bcast()`** and **`MPI_Reduce()`** for sending the initial map to all processes and for computing the final map in the root process, respectively. Also, many operations in the code involve copying data from one array to another. We could attempt to design an alternative algorithm, which makes these operations more efficient and reduces the memory footprint.

We can time the 1D version of the code using the same method described above (i.e., computing average time per step), so that we can use its performance as a baseline reference to compare apples-to-apples to the performance of the 2D version of the code.

Lastly, we assume normality of the performance data in this experiment. In the future, we can conduct diagnostic checking to test the data for normality. For instance, we can use a Q-Q plot to visually inspect the data for normality [5]. If the distribution is unknown, we can consider computing non-parametric metrics such as median and confidence interval of the median of runtimes [5].

References

- [1] D. Henty. "MPI Casestudy: Parallel Percolation Simulation", The University of Edinburgh, 2022.
- [2] B188633. "Two-Dimensional Decomposition of Percolate Message Passing Parallel Program (Part 1)", The University of Edinburgh, 2022.
- [3] RookieHPC, "MPI_Dims_create", Rookiehpc.com, 2022. [Online]. Available: https://www.rookiehpc.com/mpi/docs/mpi_dims_create.php. [Accessed: 02- May- 2022].
- [4] EPCC. Cirrus.readthedocs.io. 2022. Cirrus — Cirrus 1.2 Documentation. [Online] Available: <https://cirrus.readthedocs.io/en/main/>. [Accessed: 02- May- 2022].
- [5] T. Hoefer and R. Belli, "Scientific benchmarking of parallel computing systems: twelve ways to tell the masses when reporting performance results," SC '15: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, Austin, TX, 2015, pp. 1-12. doi: 10.1145/2807591.2807644.

Appendix A: Correctness Test Data

In this appendix, we present the tabulated results of our formal correctness tests of the percolate parallel code using two-dimensional domain decomposition.

<i>L</i>	<i>rho</i>	<i>seed</i>	<i>serial</i>			<i>parallel</i>			
			<i>actual density</i>	<i>steps</i>	<i>percolation</i>	<i>processes</i>	<i>actual density</i>	<i>steps</i>	<i>percolation</i>
768	0.4040	7777	0.404158	1903	no	1	0.404158	1700	no
768	0.4040	7777	0.404158	1903	no	2	0.404158	1648	no
768	0.4039	7777	0.404078	2546	yes	3	0.404078	2653	yes
768	0.4039	7777	0.404078	2546	yes	4	0.404078	2609	yes
384	0.4040	9999	0.403436	1056	yes	16	0.403436	1187	yes
580	0.4039	9999	0.403008	1184	yes	29	0.403008	1609	yes
768	0.0000	5555	0.000000	1152	yes	64	0.000000	1153	yes
1536	1.0000	5555	1.000000	1	no	128	1.000000	1	no

Table 1: Correctness test results using different combinations of *L*, *rho*, *seed*, and *NPROC* parameters of the parallel code, compared to the corresponding results of the serial code.

Appendix B: Performance Test Data

In this appendix, we present the tabulated results of our performance tests of the percolate parallel code using two-dimensional domain decomposition.

<i>Processes</i>	<i>L=3072</i>	<i>L=3072</i>	<i>L=3072</i>	<i>Mean</i>	<i>STDEV</i>	<i>CI.T</i>
1	0.042664	0.042646	0.042637	0.042649	0.000014	0.000034
2	0.020986	0.020759	0.019270	0.020338	0.000932	0.002316
4	0.010901	0.009652	0.009726	0.010093	0.000701	0.001741
8	0.005886	0.005855	0.005877	0.005873	0.000016	0.000040
16	0.003133	0.003133	0.003140	0.003135	0.000004	0.000010
32	0.001610	0.001636	0.001619	0.001622	0.000013	0.000033
64	0.000830	0.000836	0.000836	0.000834	0.000003	0.000009
128	0.000432	0.000428	0.000431	0.000430	0.000002	0.000005

Table 2: Time per step (in seconds) across different process counts when *L=3072*. Maximum step for this problem size was fixed at 3072 (i.e., *Lx1*).

<i>Processes</i>	<i>L=384</i>	<i>L=768</i>	<i>L=1536</i>	<i>L=3072</i>
1	0.000604	0.002379	0.009489	0.042649
2	0.000334	0.001298	0.005154	0.020338
4	0.000178	0.000692	0.002661	0.010093
8	0.000092	0.000377	0.001458	0.005873
16	0.000055	0.000207	0.000803	0.003135
32	0.000029	0.000102	0.000410	0.001622
64	0.000033	0.000069	0.000230	0.000834
128	0.000031	0.000041	0.000112	0.000430

Table 3: Average time per step (in seconds) of all tested system sizes across a range of process counts.